



Appendix A

Context Engine Reference Guide

The context engine is a sophisticated, multi-agent system designed to transform the interaction paradigm with LLMs. It moves beyond the limitations of simple prompt-response cycles, which often rely on statistical chance and produce unpredictable results. Instead, the Context Engine provides a structured environment for directed creation, enabling engineers to control not just what the AI generates, but how it reasons within defined boundaries. This approach shifts the user's role from that of a questioner to that of an architect, designing the very landscape of the AI's thought process.

This engine is the practical implementation of context engineering, the discipline of transforming generative AI into a fully controlled and reliable partner. It achieves this control through the strategic deployment of agents orchestrated by a central Planner and Executor, all communicating via a standardized **Model Context Protocol (MCP)**. The system is designed as a glass-box architecture, prioritizing transparency, predictability, and security, making it suitable for enterprise deployment.

This *Context Engine Reference Guide* serves as your technical companion, bringing together all the concepts and code patterns from the book into one clear, usable format. It translates the theory of context engineering into practical instructions for building, extending, and maintaining the system. Each section connects what was explained in earlier chapters to the actual implementation, showing how every function, agent, and helper contributes to the larger architecture. For readers working hands-on, it offers clarity on how the system operates, how data flows through each component, and how to adapt the design for real-world needs. It also serves as a long-term resource that supports further experimentation, customization, and collaboration once the core Context Engine is built.

Theoretical foundations

The core philosophy of the Context Engine is grounded in the linguistic theories that revolutionized how we understand language structure and meaning. We move away from viewing sentences merely as linear sequences of words and instead embrace them as multidimensional structures of meaning. This approach is heavily influenced by the foundational work of Lucien Tesnière and Charles J. Fillmore.

Lucien Tesnière introduced **dependency grammar**, visualizing sentences as hierarchical structures (stemmas) where words depend on other words, centering around a main verb. Charles J. Fillmore expanded upon this with his *The Case for Case*, which evolved into **semantic role labeling (SRL)**. SRL is the technique of deconstructing a sentence to answer the fundamental question: Who did what to whom, when, where, and why? ([Chapter 1](#)). It identifies the functional role each component plays in relation to the central action (the predicate).

Semantic blueprint

The Context Engine operationalizes SRL through the use of semantic blueprints. A semantic blueprint is a structured, unambiguous plan provided to the LLM, typically in JSON format. It defines the goal of the generation, the style guide, the structure, and the roles of the participants involved in the scene or topic.

By using semantic blueprints, we provide the LLM with a precise schema rather than an open-ended request. This transforms the creative act into a reliable engineering process. In the Context Engine, these blueprints are stored in a dedicated vector database (the context library) and retrieved dynamically by the Context Librarian agent based on the user's intent. The Librarian analyzes the intent of the user's goal and selects the most appropriate blueprint, which is then used by the Writer agent to ensure the final output adheres strictly to the desired structure and style. This mechanism is the cornerstone of the engine's ability to deliver predictable and goal-oriented results.

This reference guide provides a comprehensive technical overview of the Context Engine's architecture, components, and workflows. It is designed to equip context engineers with the necessary knowledge to implement, configure, and maintain the system effectively.

Merits of the glass-box architecture

The primary merit of this Context Engine lies in its glass-box architecture, offering a stark contrast to the opaque black-box systems prevalent in the market. When operating a black-box system, the engineer is often reduced to a mere consumer of an API, lacking insight into the reasoning process and having limited control over the workflow, safeguards, or knowledge retrieval mechanisms. Debugging becomes an exercise in trial-and-error prompt refinement.

In contrast, this Context Engine provides total transparency. The execution trace exposes every step of the reasoning process, allowing engineers to pinpoint exactly where a failure occurred—whether in planning, knowledge retrieval, sanitization, or generation. This modularity grants unparalleled control, enabling the customization of agents, the enrichment of the context library, and the precise tuning of safety protocols to meet specific enterprise requirements.

What makes this Context Engine unique in the current generative AI landscape is its explicit grounding in linguistic theory and its architectural approach to procedural context. While many systems use RAG for factual retrieval, this engine pioneers the use of RAG for *procedural* retrieval via the Context Librarian. By operationalizing SRL ([Chapter 1](#)) into dynamically retrieved semantic blueprints, the engine gains precise control over style, structure, and intent, moving beyond simple fact-based generation. Furthermore, the architecture uniquely acknowledges the limits of automation. It explicitly integrates the concept of a policy-driven "meta-controller," recognizing that the most robust safeguards often involve organizational rules external to the core AI reasoning process.

This holistic approach provides a mature framework for deploying verifiable, controllable, and policy-compliant AI systems.

System architecture and workflow

The Context Engine operates using a deliberate, multi-phase architecture designed for transparency, control, and safety. This architecture, finalized in [Chapter 8](#), integrates data retrieval, multi-agent reasoning, and robust safety guardrails. This document focuses on the Context Engine after the installation (downloads, modules, and libraries) process. The following flowchart (*Figure A.1*) illustrates the end-to-end process from data ingestion to final output delivery at a high functional level.

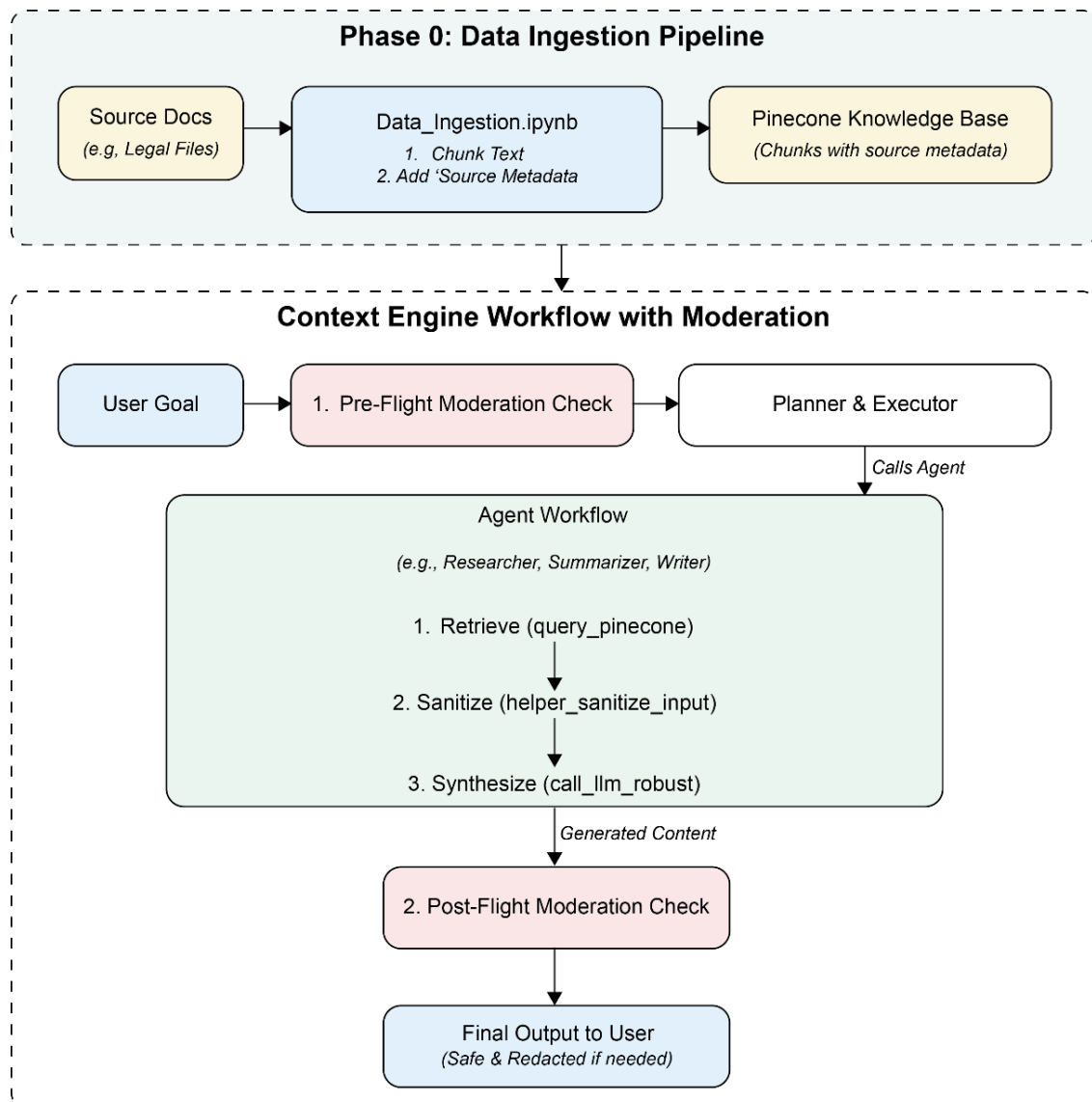


Figure A.1: The Context Engine architecture and workflow

The system operates in two main stages: the preparatory data ingestion pipeline (Phase 0) and the runtime Context Engine workflow.

Phase 0: Data ingestion pipeline

This phase prepares the data required for the engine's operation, as follows:

1. **Source docs:** Raw documents (e.g., legal files, research papers) are collected.
2. **Data ingestion** (`Data_Ingestion.ipynb`): This script processes the raw documents. It chunks the text into manageable segments and, critically, adds source metadata to each chunk. This metadata is essential for high-fidelity RAG, enabling verifiable citations ([Chapter 8](#)).
3. **Pinecone knowledge base:** The processed chunks (with embeddings and metadata) are uploaded to the vector database, ready for semantic retrieval.

Context Engine workflow

This is the runtime operation of the system, managed by the `context-engine()` function:

1. **User goal:** The workflow begins when a user submits a high-level objective.
2. **Pre-flight moderation check:** The user's goal is immediately vetted by the `helper_moderate_content` function. This is the first safety guardrail, designed to protect the system from processing inappropriate or harmful requests. If the goal is flagged, the process halts.
3. **Planner and Executor:** If the goal is safe, the Planner analyzes the goal and the available agent capabilities (provided by `AgentRegistry`). It generates a multi-step execution plan. The Executor then takes control, managing the execution of this plan.
4. **Agent workflow:** The Executor calls the appropriate agents (e.g., Librarian, Researcher, Summarizer, or Writer) sequentially as defined in the plan. The internal workflow varies by agent. For example, the Researcher's process involves the following steps:
 1. **Retrieve:** Querying the Pinecone knowledge base for relevant information (`query_pinecone`)
 2. **Sanitize:** Inspecting the retrieved data for potential prompt injection threats (`helper_sanitize_input`)
 3. **Synthesize:** Using an LLM to generate content or an answer based on the sanitized context (`call_llm_robust`)
 4. **Context chaining:** The Executor manages the state, passing the output of one agent as the input to the next using the `resolve_dependencies` mechanism
5. **Post-flight moderation check:** After the entire plan is executed and the final content is generated, the second safety guardrail is activated. The `helper_moderate_content` function vets the AI's output before it is returned to the user.

6. **Final output to user:** If the generated content is flagged, it is automatically redacted with a safe message. Otherwise, the safe and finalized content is delivered to the user.

This architecture ensures that the system remains transparent, controllable, and secure throughout its operation. The separation of concerns between the Planner, Executor, and agents, combined with the robust safety protocols, makes the Context Engine suitable for enterprise deployment.

The commons library reference

The `commons` library contains the core implementation of the Context Engine. It is organized into several Python modules, each responsible for a specific aspect of the system's functionality. This section provides a detailed reference for every function within these modules. We will begin by examining the helper functions that provide the foundational interactions with external services.

File: helpers.py

The `helpers.py` file contains essential utility functions that handle interactions with external APIs (such as OpenAI and Pinecone), data formatting, and security protocols. These functions are designed to be robust, incorporating error handling and retry logic for production environments.

`call_llm_robust()`

This centralized function handles all interactions with the LLM for content generation and planning:

- **Purpose:** To provide a standardized, resilient interface for calling the LLM, ensuring reliability through automated retries
- **Dependencies:**
 - `tenacity`: Used for the `@retry` decorator to handle transient API failures
 - `openai.APIError`: Used for specific error handling
- **Parameters:**
 - `system_prompt (str)`: The foundational instructions defining the AI's role and task constraints
 - `user_prompt (str)`: The specific input or data that the AI needs to process

- `client` (OpenAI client object): The initialized client object for API interaction (dependency injection)
- `generation_model` (`str`): The identifier of the LLM to use (e.g., `gpt-5`)
- `json_mode` (`bool`): If `True` , instructs the LLM to return the output strictly in JSON format
- **Workflow:**
 1. The function logs the attempt to call the LLM.
 2. It sets `response_format` based on the `json_mode` parameter.
 3. It calls the `client.chat.completions.create` method with the provided prompts and model.
 4. If successful, it returns the stripped content of the response.
 5. If an `APIError` error or other exception occurs, it logs the error, and the tenacity library manages the retry logic (up to six attempts with exponential backoff).

get_embedding()

This function handles the generation of vector embeddings for text strings, which is essential for semantic search operations in the vector database:

- **Purpose:** To convert a single piece of text into a numerical vector representation using the specified embedding model
- **Dependencies:**
 - `tenacity` : Used for the `@retry` decorator
 - `openai.APIError` : Used for specific error handling
- **Parameters:**
 - `text` (`str`): The input text to embed
 - `client` (OpenAI client object): The initialized client object for API interaction
 - `embedding_model` (`str`): The identifier of the embedding model to use (e.g., `text-embedding-3-small`).
- **Workflow:**
 1. The input text is cleaned by replacing newline characters with spaces, as required by the API.
 2. It calls the `client.embeddings.create` method.
 3. If successful, it returns the generated embedding vector.
 4. It utilizes the same retry logic as `call_llm_robust` for reliability.

create_mcp_message()

This function implements the MCP, the standardized communication format used between all components of the Context Engine:

- **Purpose:** To structure data into a consistent dictionary format for inter-agent communication
- **Parameters:**
 - `sender` (`str`): The name of the agent or component sending the message
 - `content` (`dict` or `str`): The primary payload of the message
 - `metadata` (`dict` , optional): Any additional contextual information
- **Structure:** The function returns a dictionary with the following keys:
 - `protocol_version` : Indicates the version of the MCP being used
 - `sender` : The sender identifier
 - `content` : The main data
 - `metadata` : Supplemental data

query_pinecone()

This function manages the interaction with the Pinecone vector database for retrieving relevant information or blueprints:

- **Purpose:** To embed a query text and perform a semantic search against a specified namespace in the Pinecone index
- **Parameters:**
 - `query_text` (`str`): The text to search for
 - `namespace` (`str`): The specific partition within the index to search (e.g., `KnowledgeStore` or `ContextLibrary`)
 - `top_k` (`int`): The number of results to return
 - `index` (Pinecone index object): The initialized index object for interaction
 - `client` (OpenAI client object): Required by the internal call to `get_embedding`
 - `embedding_model` (`str`): Required by the internal call to `get_embedding`
- **Workflow:**
 1. The function logs the query attempt.
 2. It calls `get_embedding` to convert `query_text` into a vector.
 3. It calls the `index.query` method with the vector, namespace, and `top_k`.
 4. It requests that metadata (such as the source text or blueprint JSON) be included in the response.
 5. It returns the list of matches found.

count_tokens()

This utility is used for managing context length, which is crucial for cost control and ensuring inputs do not exceed the model's limits:

- **Purpose:** To calculate the number of tokens in a given text string based on a specific model's encoding scheme
- **Dependency:** `tiktoken` (the library used for token counting)
- **Parameters:**
 - `text (str)`: The input text to analyze
 - `model (str)`: The model identifier to determine the correct encoding
- **Workflow:**
 1. It attempts to load the specific encoding for the provided model.
 2. If the model is not found, it falls back to the standard `cl100k_base` encoding.
 3. It encodes the text and returns the length of the resulting token list.

`helper_sanitize_input()`

This function is a critical security component introduced to mitigate the risk of prompt injection attacks ([Chapter 7](#)):

- **Purpose:** To detect and block known patterns associated with attempts to manipulate the LLM's instructions
- **Dependency:** `re` (the regular expression module)
- **Parameter:** `text (str)`: The input text to sanitize (typically data retrieved from the vector database)
- **Workflow:**
 1. The function defines an `injection_patterns` list (e.g., `ignore previous instructions`, or `act as`).
 2. It iterates through the patterns and uses `re.search` (case-insensitive) to check whether the pattern exists in the input text.
 3. If a pattern is found, it logs a warning and raises a `ValueError` error, immediately halting the processing of that specific piece of text.
 4. If no patterns are found, it logs a success message and returns the original text.

`helper_moderate_content()`

This function implements the automated content moderation shield, ensuring that the engine operates within responsible boundaries ([Chapter 8](#)):

- **Purpose:** To use the OpenAI Moderation API to check whether the content (either user input or AI output) violates safety policies
- **Parameters:**
 - `text_to_moderate ( str )`: The content to be vetted
 - `client` (OpenAI client object): The initialized client object for API interaction
- **Workflow:**
 1. The function logs the moderation attempt.
 2. It calls `client.moderations.create` with the input text.
 3. It parses the API response to build a comprehensive report containing the following:
 1. `flagged ( bool )`: Whether any policy was violated
 2. `categories ( dict )`: Which specific categories were violated
 3. `scores ( dict )`: The confidence scores for each category
 4. If the content is flagged, it logs a warning.
 5. Crucially, it includes a "fail safe" mechanism: if the API call fails for any reason (e.g., network error), it returns a report indicating the content is flagged, preventing unvetted content from proceeding.

We have now detailed the foundational utilities in the `helpers.py` file. Next, we will examine the specialist agents that utilize these utilities to perform specific tasks.

File: agents.py

The `agents.py` file defines the specialist agents that perform the core tasks within the Context Engine. Each agent is designed to handle a specific responsibility, such as retrieving instructions, finding facts, or generating content.

agent_context_librarian()

The Context Librarian is responsible for retrieving the appropriate semantic blueprint based on the desired style or structure of the output:

- **Purpose:** To perform a semantic search against the Context Library to find the blueprint that matches the user's intent.
- **Parameters:**
 - `mcp_message ( dict )`: The input message containing the required `intent_query`

- `client`, `index`, and `embedding_model`: Dependencies passed for `query_pinecone`
- `namespace_context (str)`: The Pinecone namespace containing the semantic blueprints
- **Workflow:**
 1. The agent extracts `intent_query` from the `mcp_message` input.
 2. It calls `query_pinecone` to search `namespace_context` (context library). The embeddings in this namespace correspond to the descriptions of the blueprints.
 3. If a match is found, it retrieves the `blueprint_json` from the metadata and returns it in an MCP message.
 4. If no match is found, it returns a default, neutral blueprint.

agent_researcher()

The Researcher agent is responsible for retrieving and synthesizing factual information from the knowledge store. This implementation is upgraded for high-fidelity RAG, ensuring that source citations are included (**Chapter 8**):

- **Purpose:** To find relevant data chunks, sanitize them, and synthesize a factual answer with citations
- **Parameters:**
 - `mcp_message (dict)`: The input message containing the required `topic_query`
 - `client`, `index`, `generation_model`, and `embedding_model`: Dependencies for `query_pinecone` and `call_llm_robust`
 - `namespace_knowledge (str)`: The Pinecone namespace containing the factual data
- **Workflow:**
 1. The agent extracts `topic_query` from the `mcp_message` input.
 2. It calls `query_pinecone` to retrieve the top three relevant chunks from `namespace_knowledge`.
 3. It iterates through the results, calling `helper_sanitize_input` on the text of each chunk. If a chunk fails sanitization, it is skipped.
 4. It collects the unique source document names from the metadata of the sanitized chunks.
 5. It constructs a prompt that includes the sanitized source material and the topic, instructing the LLM (via `call_llm_robust`) to synthesize an answer based *only* on the provided sources.

6. It combines the LLM's findings with the programmatically collected sources and returns `answer_with_sources` in an MCP message.

agent_writer()

The Writer agent is the final generation component, responsible for applying the instructions from a semantic blueprint to the factual source material:

- **Purpose:** To generate the final output by combining research findings with stylistic and structural instructions
- **Parameters:**
 - `mcp_message (dict)`: The input message containing the blueprint, and either facts or `previous_content`
 - `client` and `generation_model`: Dependencies for `call_llm_robust`
- **Workflow:**
 1. The agent unpacks the blueprint, facts, and `previous_content` from the `mcp_message` input.
 2. It implements logic to handle various data contracts for facts (e.g., `facts`, `summary`, and `answer_with_sources`) to ensure compatibility with different upstream agents (Researcher or Summarizer).
 3. It determines the source material (either new facts or content for rewriting).
 4. It constructs a prompt combining the semantic blueprint and the source material.
 5. It calls `call_llm_robust` to generate the final content according to the blueprint.
 6. It returns the final output in an MCP message.

agent_summarizer()

The Summarizer agent is introduced for context reduction, acting as a gatekeeper to manage token counts and costs:

- **Purpose:** To condense a large text into a concise summary guided by a specific objective
- **Parameters:**
 - `mcp_message (dict)`: The input message containing `text_to_summarize` and `summary_objective`
 - `client` and `generation_model`: Dependencies for `call_llm_robust`
- **Workflow:**
 1. The agent extracts `text_to_summarize` and `summary_objective` from the input.

2. It constructs a prompt instructing the LLM to summarize the text according to the objective.
3. It calls `call_llm_robust` to perform the summarization.
4. It returns the summary in an MCP message.

We have now documented the specialist agents responsible for executing tasks. Next, we will examine the mechanism that organizes and deploys these agents.

File: registry.py

The `registry.py` file implements the agent registry, which acts as the central directory and factory for the specialist agents. It allows the Planner to understand the available capabilities and the Executor to invoke the correct agent handler.

Class: AgentRegistry

This class manages the registration and retrieval of agent functions:

- Initialization (`__init__`)
- The constructor initializes the `self.registry` dictionary, mapping agent names (e.g., `Librarian`) to their corresponding functions in the `agents.py` module (e.g., `agents.agent_context_librarian`)

Method: get_handler()

This is the core factory method used by the Executor:

- **Purpose:** To retrieve the appropriate handler function for a given agent name and inject the necessary runtime dependencies
- **Parameters:**
 - `agent_name (str)`: The name of the agent requested by the Planner
 - `client`, `index`, `generation_model`, `embedding_model`, `namespace_context`, and `namespace_knowledge`: The runtime configuration and initialized clients
- **Workflow:**
 1. It looks up `agent_name` in the registry. If not found, it raises `ValueError`.
 2. It uses conditional logic to determine which dependencies the specific agent requires.

3. It returns a lambda function that wraps the original agent function. This lambda accepts `mcp_message` as input and calls the underlying agent function with the pre-injected dependencies. This ensures that agents only receive the parameters they need.

Method: `get_capabilities_description()`

This method is crucial for the planning phase of the Context Engine:

- **Purpose:** To generate a structured, human-readable description of all available agents, their roles, and their required input parameters.
- **Workflow:** The method returns a multi-line string formatted specifically to be embedded into the Planner's system prompt. This description provides the Planner LLM with the necessary *toolkit* information to create a valid execution plan, ensuring it uses the correct agent names and input keys.

Global object: `AGENT_TOOLKIT`

An instance of `AgentRegistry` is initialized globally when the module is imported. This `AGENT_TOOLKIT` object is used by the Context Engine (`engine.py`) during the planning and execution phases.

The agent registry provides the necessary abstraction layer for managing the system's capabilities. We will now examine the core engine that utilizes this registry to orchestrate the entire workflow.

File: `engine.py`

The `engine.py` file contains the core logic of the Context Engine, encompassing the Tracer, the Planner, and the Executor. This file orchestrates the entire process from receiving a goal to delivering the final output.

Class: `ExecutionTrace`

The `ExecutionTrace` class provides essential debugging and observability capabilities by logging the entire execution flow:

- **Purpose:** To maintain a detailed record of the plan, the inputs and outputs of each step, the execution status, and the duration
- **Methods:**

- `__init__ ( self , goal )`: Initializes the trace, records the goal, and sets the start time
- `log_plan ( self , plan )`: Records the execution plan generated by the Planner
- `log_step ( self , step_num , agent , planned_input , mcp_output , and resolved_input )`: Logs the details of a single execution step, crucially capturing both the planned input (with placeholders) and the actual resolved input (with references replaced by data)
- `finalize ( self , status , and final_output=None )`: Finalizes the trace, records the status and the final output, and calculates the total duration

Function: `planner()`

The Planner is the strategic core of the engine, responsible for interpreting the goal and generating a structured execution plan:

- **Purpose:** To use an LLM to create a step-by-step plan utilizing the available agent capabilities
- **Parameters:**
 - `goal ( str )`: The high-level objective provided by the user
 - `capabilities ( str )`: The description of agents generated by `AgentRegistry.get_capabilities_description()`
 - `client` and `generation_model`: Dependencies for `call_llm_robust`
- **Workflow:**
 1. It constructs a detailed system prompt that includes the available capabilities and strict instructions for generating a JSON-formatted plan.
 2. It emphasizes the use of context chaining (using `$$STEP_N_OUTPUT$$` references for dependencies).
 3. It calls `call_llm_robust` with `json_mode=True` to generate the plan.
 4. It parses the JSON string and validates that it conforms to the expected `{"plan": [...]}` structure.
 5. If the structure is valid, it returns the plan (a list of steps). Otherwise, it raises an error.

Function: `resolve_dependencies()`

This helper function is critical for implementing context chaining during the execution phase:

- **Purpose:** To replace the placeholder dependency references (e.g., `$$STEP_1_OUTPUT$$`) in a step's input parameters with the actual data generated by previous steps

- **Parameters:**
 - `input_params` (dict): The input parameters for the current step as defined by the Planner
 - `state` (dict): The execution state containing the outputs of all previous steps
- **Workflow:** The function uses recursion to deeply traverse the `input_params` dictionary. If it encounters a string value matching the dependency reference format, it looks up the corresponding key in the state. If the key is found, it replaces the reference with the data; otherwise, it raises `ValueError`, indicating a dependency error in the plan.

Function: `context_engine()`

This is the main entry point and the Executor of the Context Engine:

- **Purpose:** To manage the entire lifecycle of a task, including planning, execution, state management, and tracing
- **Parameters:**
 - `goal` (str): The user's objective
 - `client`, `pc` (Pinecone client), and `index_name`: Initialized clients and configuration
 - `generation_model`, `embedding_model`, `namespace_context`, and `namespace_knowledge`: Runtime configuration parameters
- **Workflow:**
 1. **Initialization:** Initializes `ExecutionTrace` and gets the `AgentRegistry` instance. Connects to the Pinecone index.
 2. **Phase 1 – Plan:**
 1. Retrieves the capabilities description from the registry.
 2. Calls the `planner()` function to generate the execution plan.
 3. Logs the plan to the trace.
 3. **Phase 2 – Execute:**
 1. Initializes an empty state dictionary.
 2. Iterates through each step in the plan.
 3. **Dependency resolution:** Calls `resolve_dependencies()` to replace placeholders with data from the state.
 4. **Agent invocation:** Calls `registry.get_handler()` to retrieve the appropriate agent function with injected dependencies.
 5. Executes the agent handler with the resolved input.
 6. **State management:** Stores the output of the step in the state dictionary (e.g., `state["STEP_1_OUTPUT"] = output_data`).
 7. **Tracing:** Logs the completed step to the trace.

4. **Error handling:** Includes robust `try / except` blocks during initialization, planning, and execution. If an error occurs at any stage, it finalizes the trace with a `Failed` status and returns `None` along with the trace.
5. **Finalization:**
 1. Retrieves the output of the final step.
 2. Finalizes the trace with a `Success` status.
 3. Returns the final output and the completed trace object.

Module: `utils.py`

The `utils.py` module contains setup and configuration utilities specifically designed for the operational environment (e.g., Google Colab or a local Jupyter server):

- **`install_dependencies()`:**
 - **Purpose:** To install the required Python packages with specific versions, ensuring reproducibility and stability
 - **Implementation details:**
 - Uses the subprocess module to execute `pip install` commands
 - Specifies exact versions for key libraries (e.g., `openai`, `pinecone`, or `tenacity`)

- **initialize_clients():**
 - **Purpose:** To securely load API keys and initialize the necessary external service clients (OpenAI and Pinecone)
 - **Implementation details:**
 - Attempts to load API keys (`API_KEY` for OpenAI, `PINECONE_API_KEY`) securely, typically using an environment variable manager or a secrets manager (such as `google.colab.userdata`)
 - Sets the `OPENAI_API_KEY` environment variable
 - Initializes and returns the `OpenAI()` client and the `Pinecone()` client objects
 - Includes error handling if secrets are not found

Data ingestion pipeline

The data ingestion pipeline (implemented in `Data_Ingestion.ipynb`) is responsible for processing raw source documents and loading them into the vector database. This pipeline is designed for high-fidelity RAG (**Chapter 8**), ensuring that every piece of information stored can be traced back to its origin.

Pipeline steps

The ingestion process involves several key steps, ensuring that data is clean, correctly structured, and efficiently stored:

1. **Load documents:** The pipeline reads raw text files from a specified directory (e.g., `legal_documents`).
2. **Chunking** (the `chunk_text` function):
 1. The documents are split into smaller segments.
 2. The pipeline uses `tiktoken` for token-aware chunking, ensuring that chunks respect the token limits of the embedding model.
 3. It applies a default chunk size (e.g., 400 tokens) and also employs an overlap strategy (e.g., 50 tokens) between chunks to maintain semantic context across boundaries.
3. **Embedding** (`get_embeddings_batch` function):
 1. Chunks are sent to the embedding model (e.g., `text-embedding-3-small`) in batches to generate vectors.
 2. This function includes retry logic for robustness.
4. **High-fidelity metadata enrichment:**

1. This is the critical step for verifiability. Before uploading, the pipeline constructs the metadata for each vector.
2. It includes the raw text of the chunk.
3. It includes the source (the filename of the original document).
5. **Vector upsert (batch processing):** The vectors, along with their enriched metadata, are upserted into the Pinecone index within the designated namespace (e.g., `KnowledgeStore`).

Ingesting the context library

In addition to the knowledge base, the pipeline also ingests the semantic blueprints into the `ContextLibrary` namespace:

1. **Define blueprints:** Blueprints are defined as a list of dictionaries, each containing an ID, a description (the intent), and the blueprint (the JSON structure).
2. **Embedding:** The description of the blueprint is embedded. This allows the Librarian agent to perform a semantic search based on intent.
3. **Metadata:** `blueprint_json` itself is stored as metadata.
4. **Upsert:** The vectors are upserted into the `ContextLibrary` namespace.

Execution and operation

The Context Engine is operated through a standardized interface typically implemented within a Jupyter notebook (e.g., `Legal_assistant_Explorer.ipynb`). This interface is known as the control deck.

Engine room

The core of the operational notebook is the `execute_and_display` function. This function orchestrates the execution and integrates the safety protocols:

- **Function:** `execute_and_display()`
- **Purpose:** To serve as the main wrapper for running `context_engine`, handling moderation checks, and displaying the results and trace
- **Signature:**

```
execute_and_display(  
    goal, config, client, pc, moderation_active=False)
```

- **Workflow:**

1. **Pre-flight moderation:** If `moderation_active` is `True`, it calls `helper_moderate_content` on the goal. If flagged, execution stops.
2. **Execution:** It calls the main `context_engine` function, passing the goal and configuration.
3. **Post-flight moderation:** If `moderation_active` is `True` and a result was produced, it calls `helper_moderate_content` on the result. If flagged, the result is redacted.
4. **Display output:** It displays the final (potentially redacted) result using Markdown formatting.
5. **Display trace:** It displays `ExecutionTrace` (status, duration, and detailed steps) for debugging and analysis.

Control deck

The control deck is the interactive cell where the context engineer defines the task and configuration for a specific run.

The workflow of the control deck is as follows:

1. **Define the goal:** The high-level objective provided by the user.
2. **Define configuration:** A dictionary specifying the operational parameters (models, index name, and namespaces).
3. **Execute:** Call the execution function, optionally activating moderation.

The mature architecture allows for the creation of generic templates for common workflow patterns.

- **Template 1 – high-fidelity RAG:** Used for tasks requiring verifiable research and citations. The Planner typically orchestrates the Librarian, Researcher, and Writer.
- **Template 2 – context reduction:** Used for tasks involving large documents. The Planner typically chains the Summarizer to the Writer to manage token counts efficiently.
- **Template 3 – grounded reasoning:** Used to validate that the engine does not hallucinate when the required information is absent from the knowledge base.

Production safeguards: moderation, sanitization, and policy

A production-ready AI system requires robust safeguards to ensure reliability, predictability, and security. The Context Engine implements a multi-layered defense strategy involving input sanitization and a two-stage content moderation protocol. However, the ultimate safeguard lies in integrating the AI within clear, human-defined organizational policies ([Chapter 8](#)).

Input sanitization (prompt injection defense)

The input sanitization layer is designed to mitigate the risk of prompt injection, where malicious text hidden within retrieved data attempts to manipulate the LLM's instructions:

- **Mechanism:** The `helper_sanitize_input()` function, implemented within `agent_researcher()`
- **Implementation details:**
- After data is retrieved from the knowledge base and before it is sent to the LLM for synthesis, every chunk of text is vetted against known injection patterns (e.g., `ignore previous instructions`)
- If a threat pattern is detected, the tainted chunk is skipped entirely

Two-stage content moderation protocol

The content moderation protocol acts as an automated shield using the OpenAI Moderation API via the `helper_moderate_content()` function:

- **Stage 1 – Pre-flight check (input vetting):**
 - **Purpose:** To protect the system from processing inappropriate user requests.
 - **Workflow:** The user's goal is analyzed before execution begins. If flagged, the entire process is halted.
- **Stage 2 – Post-flight check (output vetting):**
 - **Purpose:** To protect the user from seeing potentially harmful content generated by the AI.
 - **Workflow:** The AI's final generated output is analyzed. If flagged, the output is automatically redacted with a standard safe message.

Limits of automation and the role of policy

While sanitization and moderation are powerful tools, they encounter "reality factors" where purely technical solutions fail. Examples include legitimate profanity in witness testimonies or complex documents where context determines appropriateness (**Chapter 8**).

An AI cannot intuit organizational rules (e.g., that profanity is allowed in a legal quote but not in an email body). Attempting to solve this with increasingly complex code inside the engine leads to unmanageable systems.

Policy-driven solution

The true solution is architectural and organizational. The context engineer must recognize that the system includes the entire ecosystem, including business processes and human users. The final safeguard is a **meta-context engine controller**.

This is a higher-level application that sits *above* the core Context Engine. It is responsible for the following:

- **Input parsing:** Handling messy, real-world input (e.g., separating email bodies from attachments)
- **Policy enforcement:** Executing deterministic, human-defined business rules established through organizational workshops (e.g., applying different moderation rules based on the input source)
- **Control Deck assembly:** Assembling a clean, safe, and unambiguous control deck (goal, config, and hyperparameters such as `moderation_active`) for the Context Engine

This architecture separates deterministic business logic from the non-deterministic AI reasoning, ensuring the system remains robust, maintainable, and aligned with the organization's intent.

Operational realities: latency and stochasticity

As you deploy the Context Engine, you will observe that executing complex goals is not instantaneous. This latency should not be mistaken for an error or inefficiency; rather, it is the tangible manifestation of the

deep thinking process we have architected. Advanced reasoning frameworks, such as Google Gemini 2.5 Pro Ultra at the time of the writing of this book, exhibit similar characteristics, utilizing extensive computational resources and recurrent processes to achieve high-quality, nuanced results. While the Context Engine operates on a much smaller scale, it inherits this deliberate methodology. It breaks down goals into multi-step plans, requiring sequential API calls for planning, embedding, database retrieval, research synthesis, and generation. This cascade of operations takes time. The context esngineer must understand that this deliberate pace is the necessary trade-off for the reliable, transparent, and high-fidelity reasoning the system provides.

It is also crucial to remember that the LLM at the heart of the engine remains a stochastic system. LLMs operate on probabilities, generating the most likely sequence of tokens rather than following a deterministic path. The Context Engine provides a powerful framework for directing this process, using semantic blueprints and context chaining to significantly constrain the output and ensure goal alignment. However, it does not eliminate the inherent probabilistic nature of the underlying model. As such, the context engineer should expect variations in the exact phrasing, tone, or structure of the output between different runs, even when the input goal remains identical. This variability is the standard operational characteristic of generative AI, not a failure of the engine's control mechanisms.

Table of contents collapsed